

BrainChip AKD1000

Manual de puesta en marcha, ejecución de modelos y diagnóstico



Álvaro Poveda Ruiz

Material complementario del Trabajo Fin de Grado
Universidad Politécnica de Madrid, 2026

Versión 1.2.2 · Revisión: 29 de septiembre de 2026

Resumen

Este manual describe la puesta en marcha de una tarjeta BrainChip AKD1000 conectada por PCIe. El procedimiento incluye la preparación del equipo, la instalación del controlador y del entorno de Python, la carga de un modelo y la comprobación de su ejecución. También explica cómo medir la latencia y localizar los fallos más habituales.

La primera prueba utiliza un modelo sencillo cuya salida se conoce de antemano. Se ejecuta primero en software y después en la tarjeta. Así se puede comprobar por separado el funcionamiento del entorno de Akida, la compatibilidad del modelo y el acceso al hardware antes de utilizar una aplicación propia.

El montaje empleado en el TFG se recoge en el apéndice A, junto con sus versiones y resultados. Las herramientas incluidas permiten repetir la prueba básica, pero no reproducen por sí solas el experimento AoA: sus modelos y datos no forman parte de esta entrega. La ejecución completa de esta versión del procedimiento en una placa física sigue pendiente.

Alcance del manual

El manual cubre las operaciones necesarias para instalar el entorno, detectar la tarjeta, cargar y ejecutar modelos, consultar las estadísticas y medir el rendimiento. También incluye comprobaciones para localizar errores de conexión, controlador, datos o modelo.

Las instrucciones de montaje deben aplicarse a la revisión concreta de cada componente. Antes de conectar la alimentación, hay que comprobar la tensión, la polaridad y la orientación del cable en la documentación del fabricante. El manual no cubre la reparación electrónica de la tarjeta ni su diseño interno.

Cómo usar este documento

Descarga el PDF junto con `akd1000-bringup.zip` y extrae el ZIP. Abre una terminal en la carpeta `akd1000-bringup`, donde están `pyproject.toml`, `examples/` y `models/`. Todos los comandos del proyecto parten de esa carpeta, salvo que se indique otra. No ejecutes los ejemplos desde el visor del ZIP. Conserva el PDF y el ZIP con la misma versión.

Los recuadros azules indican decisiones; los amarillos, precauciones; los rojos, condiciones de parada; y los verdes, datos del TFG. Ejecuta solo el bloque que corresponda a tu equipo. Los bloques que usan `sudo`, `apt` o `lspci` son para Linux. El [capítulo 4](#) incluye la activación del entorno en Windows.

Si trabajas sin tarjeta, empieza por el [capítulo 4](#) y completa la prueba en software. Si dispones del hardware, lee primero el inventario y el montaje de los [capítulos 1](#) y [2](#); después prepara el controlador y el SDK.

Versión y material incluido

Campo	Valor
Versión y fecha	1.2.2; 29 de septiembre de 2026
Fuente editable	<code>manual/main.tex</code> y <code>manual/sections/</code>
Referencia del TFG	Python 3.11.2 y Akida 2.19.1; apéndice A
Pruebas de esta edición	Pruebas automatizadas y simulador con Akida 2.19.1 y 2.19.3 en Windows; ejecución física pendiente
Modelo incluido	<code>models/akida_v1_smoke_test.fbz</code> ; cuatro entradas y dos salidas, sin datos AoA

Las referencias finales contienen enlaces a las instrucciones oficiales. La ruta de software actual y la compatibilidad del controlador se comprueban por separado: que un SDK se instale no garantiza que el kernel pueda cargar el controlador PCIe.

Índice

Resumen	2
Cómo usar este documento	3
1 Orientación y elección de ruta	7
1.1 Qué componentes intervienen	7
1.2 Tres decisiones antes de empezar	7
1.3 Matriz de rutas	8
1.4 Criterios de avance	8
2 Hardware, alimentación y montaje	9
2.1 Inventario del montaje	9
2.2 Topologías cubiertas	9
2.2.1 Host Linux con ranura PCIe	9
2.2.2 Raspberry Pi 5 con adaptador FFC a PCIe	10
2.3 Montaje	10
2.4 Carrier Waveshare utilizado en el TFG	11
2.5 Comprobación antes del encendido	12
3 Preparación del host y controlador PCIe	13
3.1 Comprobaciones iniciales	13
3.2 Habilitar PCIe en Raspberry Pi 5	13
3.3 Comprobar la conexión PCIe	13
3.4 Instalar el controlador	14
3.5 Comprobar el controlador	14
4 Entorno de software y primera prueba	16
4.1 Preparar el entorno	16
4.2 Instalar Akida	16
4.3 Primera prueba en software	17
4.4 Comprobar el equipo	17
4.5 Primera ejecución en la AKD1000	17
5 Operación de modelos Akida	19
5.1 Cargar e inspeccionar un modelo	19
5.2 Seleccionar el dispositivo	19
5.3 Mapear el modelo en la tarjeta	19
5.4 Preparar la entrada	20
5.5 Ejecutar el modelo	20
5.6 Modo de reloj	21
6 Medida de rendimiento y energía	22
6.1 Qué se está midiendo	22
6.2 Medida de latencia	22
6.3 Ejecutar un benchmark	22

6.4	Potencia y energía	23
6.5	Guardar los resultados	23
7	Diagnóstico	25
7.1	Información útil para diagnosticar un fallo	26
8	Mantenimiento y actualización	27
8.1	Guardar una configuración conocida	27
8.2	Actualizar el entorno	27
8.3	Apagado y almacenamiento	27
A	Caso documentado del TFG	28
A.1	Montaje utilizado	28
A.2	Entorno de software	29
A.3	Modelos AoA	29
A.4	Resultados	30
B	Referencia rápida de comandos	31
B.1	Comprobar el sistema	31
B.2	Raspberry Pi 5	31
B.3	Controlador PCIe	31
B.4	Entorno de Python	32
B.5	Pruebas básicas	32
B.6	Ejecutar un modelo propio	32
B.7	Benchmark	33
B.8	Diagnóstico rápido	33
C	Fuentes y limitaciones	34

Índice de figuras

2.1	Montaje Raspberry Pi 5, carrier y AKD1000	10
2.2	Montaje alimentado	11
6.1	Frontera de la medida de latencia	22
A.1	Resultados de los modelos AoA	30

Índice de cuadros

1.1	Responsabilidad de cada capa del sistema.	7
1.2	Recorridos cubiertos por el manual.	8
2.1	Datos básicos del montaje.	9
7.1	Guía rápida de diagnóstico.	25
A.1	Hardware utilizado durante el TFG.	28
A.2	Entorno de software del TFG.	29
A.3	Modelos AoA utilizados en el TFG.	29

1 Orientación y elección de ruta

1.1 Qué componentes intervienen

La AKD1000 no se maneja como una aplicación aislada. El host ejecuta el sistema operativo y el runtime; PCIe transporta órdenes y datos; el controlador expone el dispositivo al sistema; y el modelo FBZ define la red que el runtime puede mapear. Separar estas capas evita buscar un fallo de alimentación en Python o intentar resolver un paquete ausente reinstalando el controlador.

Cuadro 1.1: Responsabilidad de cada capa del sistema.

Capa	Qué hace	Qué no demuestra por sí sola
Host	Ejecuta Linux o Windows, Python, preparación de datos y registros.	Que el cálculo se haya realizado en la tarjeta.
Interfaz PCIe	Transporta configuración, entradas y salidas.	Que exista un controlador utilizable.
Controlador Linux	Crea el acceso al dispositivo, normalmente mediante <code>/dev/akida*</code> .	Que el modelo sea compatible.
Runtime Akida	Carga FBZ, descubre dispositivos, mapea y ejecuta.	Que un mapeo virtual sea una inferencia física.
AKD1000	Ejecuta las secuencias asignadas al hardware.	El tiempo total de Python ni el consumo de toda la plataforma.
FBZ	Conserva una red Akida serializada.	El significado de las entradas, las etiquetas o el dataset.

La tarjeta PCIe documentada por BrainChip contiene un AKD1000, usa una interfaz host PCIe y dispone de memoria LPDDR4 accesible mediante DMA [2]. El product brief del SoC describe veinte núcleos de procesamiento neuronal y un endpoint PCIe 2.1 de dos líneas [1]. Estas son características del producto; no son una promesa de latencia, energía o utilización para cualquier red.

1.2 Tres decisiones antes de empezar

Decisión 1: software o hardware

Si aún no tienes una tarjeta, sigue la ruta **S**: entorno software, modelo de prueba y dispositivo virtual. Si tienes la tarjeta y una conexión PCIe documentada, completa primero la ruta **S** y continúa con la ruta **H**. La simulación confirma el modelo y el paquete; no confirma el cableado, el controlador ni la placa.

Decisión 2: entorno actual o de referencia

Para aprender con la documentación vigente usa la ruta **A**: MetaTF 2.19.3, o la versión que figure en la página oficial cuando leas el manual. Para repetir la prueba incluida usa la ruta **R**: Python 3.11 y Akida 2.19.1. Esta ruta no reproduce el experimento AoA completo. No mezcles ambas instalaciones en el mismo entorno virtual.

Decisión 3: tipo de host

Un PC Linux con ranura PCIe y una Raspberry Pi 5 con adaptador FFC requieren montajes distintos. Windows está incluido en las configuraciones software actuales de MetaTF [5], pero este manual no afirma soporte de la tarjeta física en Windows: las instrucciones del controlador PCIe público consultado son para Linux [3].

1.3 Matriz de rutas

Cuadro 1.2: Recorridos cubiertos por el manual.

Ruta	Equipo	Objetivo	Evidencia final
S	Windows o Linux compatible, sin tarjeta	Instalar MetaTF, importar Akida y ejecutar en software.	Versión registrada, salida del modelo y mapeo virtual.
H-PC	Host Linux, tarjeta y conexión PCIe documentada	Detectar y ejecutar en AKD1000 mediante el controlador público.	<code>lspci</code> , módulo, nodo, <code>akida devices</code> y prueba física.
H-Pi	Raspberry Pi 5, adaptador FFC y tarjeta	Habilitar PCIe cuando proceda y completar H-PC.	Configuración de arranque, enumeración y prueba física.
R	Python 3.11 y SDK de referencia	Repetir la prueba básica con Akida 2.19.1.	Salida conocida y mapeo virtual comprobado.

1.4 Criterios de avance

Cada etapa responde a una pregunta distinta:

1. **Inventario:** ¿sé exactamente qué host, tarjeta, adaptador, fuentes, sistema, kernel y arquitectura tengo?
2. **Montaje:** ¿la cadena está conectada según la documentación de esas revisiones?
3. **Enumeración:** ¿el host ve un endpoint PCIe nuevo?
4. **Controlador:** ¿el módulo se ha compilado para este kernel y existe acceso al nodo?
5. **SDK:** ¿el intérprete activo importa la versión prevista de Akida?
6. **Modelo:** ¿el FBZ carga, tiene el contrato esperado y mapea como se pretende?
7. **Ejecución:** ¿una entrada conocida produce una salida conocida en hardware?
8. **Medida:** ¿la ventana, las unidades y la frontera de potencia están declaradas?

No se salta una respuesta negativa. Registrar el primer punto que falla es más útil que ejecutar una lista larga de comandos y quedarse solo con el último error.

2 Hardware, alimentación y montaje

2.1 Inventario del montaje

Antes de empezar, anota los datos que puedas comprobar de cada componente. No completes una referencia a partir de una fotografía o por parecido con otro modelo. Si algún dato, como la revisión de una placa o su tensión de alimentación, no puede confirmarse con la etiqueta, la factura o la documentación del fabricante, indícalo como *no verificado*.

Cuadro 2.1: Datos básicos del montaje.

Elemento	Datos	Fuente
Host	Fabricante, modelo, RAM y revisión	Equipo, firmware o factura
Sistema	Distribución, versión y arquitectura	<code>/etc/os-release</code> , <code>uname -m</code>
Kernel	Versión completa	<code>uname -r</code>
Tarjeta Akida	Modelo y revisión	Etiqueta y documentación de BrainChip
Adaptador	Fabricante, modelo y tipo de enlace	Serigrafía y documentación del fabricante
Cable	Tipo y orientación	Marcado y documentación del adaptador
Fuente del host	Tensión y potencia	Etiqueta de la fuente
Fuente de la tarjeta o carrier	Tensión, polaridad y conector	Etiqueta y documentación del fabricante

Los conectores M.2, Mini PCIe, PCIe convencional y el conector FFC de Raspberry Pi 5 no son equivalentes. Comprueba siempre el tipo de conexión y la orientación del cable en la documentación del equipo utilizado.

2.2 Topologías cubiertas

2.2.1 Host Linux con ranura PCIe

En un PC Linux, la tarjeta se instala en una ranura PCIe compatible y se alimenta según las indicaciones de su documentación. El equipo debe estar apagado y desconectado antes de insertar o retirar la tarjeta.

Después del montaje, continúa con el [capítulo 3](#). Si `lspci` no detecta un nuevo dispositivo PCIe, el problema debe resolverse antes de continuar con la configuración de Python.

2.2.2 Raspberry Pi 5 con adaptador FFC a PCIe

Raspberry Pi 5 dispone de una interfaz PCIe Gen 2 $\times 1$ accesible mediante un conector FFC [7]. Para utilizar una tarjeta PCIe convencional es necesario un adaptador que lleve este enlace hasta una ranura compatible.

Aunque el adaptador disponga de una ranura físicamente más larga, el enlace proporcionado por Raspberry Pi 5 sigue siendo $\times 1$.

El montaje del TFG se muestra en la figura 2.1.

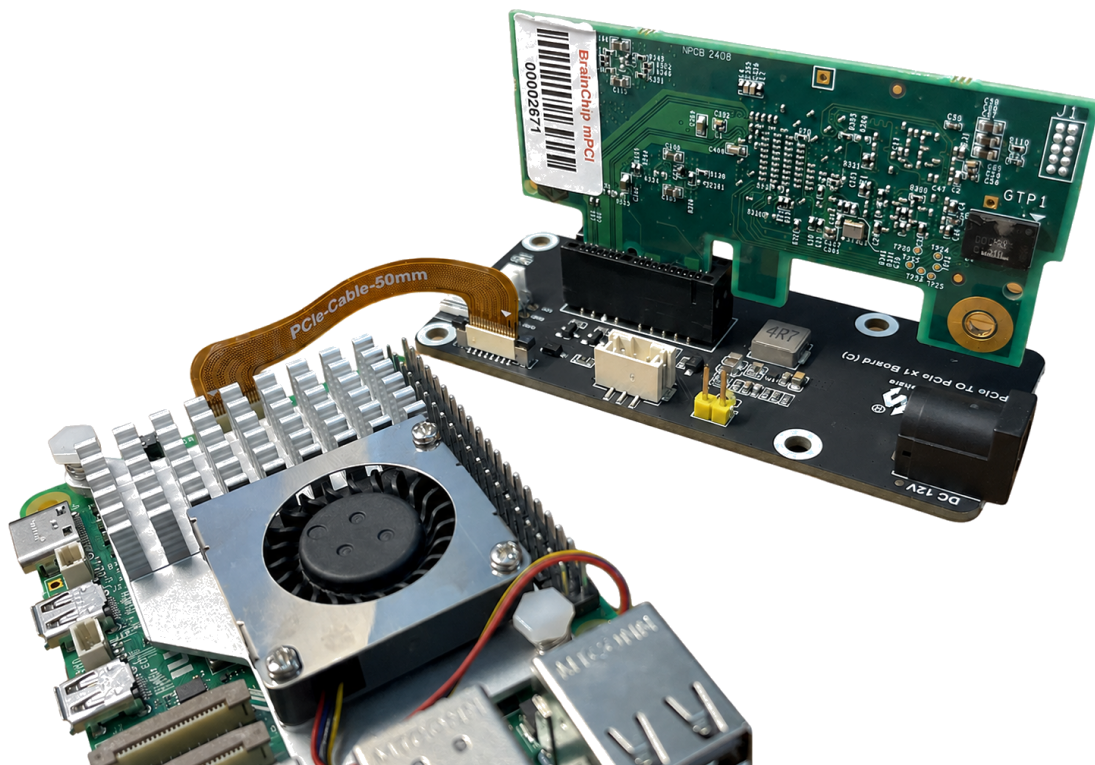


Figura 2.1: Montaje formado por una Raspberry Pi 5, un adaptador PCIe y la tarjeta BrainChip AKD1000 utilizado durante el TFG.

2.3 Montaje

Si la documentación del fabricante indica una secuencia de conexión diferente, sigue ese procedimiento.

1. Apaga el host y desconecta las fuentes de alimentación.
2. Comprueba la orientación del cable FFC y la tensión de alimentación del adaptador o de la tarjeta.
3. Conecta el cable FFC al adaptador respetando la orientación indicada.
4. Inserta la AKD1000 completamente en la ranura PCIe del carrier.
5. Revisa las conexiones antes de alimentar de nuevo el sistema.

Si aparece algún problema eléctrico

Desconecta la alimentación si aparece humo, olor anormal, una temperatura excesiva o cualquier otro comportamiento que indique un posible problema de alimentación o montaje.

2.4 Carrier Waveshare utilizado en el TFG

En el montaje del TFG se utilizó un adaptador Waveshare conectado a la Raspberry Pi 5 mediante un cable FFC de 50 mm. En la placa puede leerse DC 12V y *PCIe TO PCIe x1 Board (C)*.

La documentación actual de Waveshare describe este modelo con una ranura compatible mecánicamente con tarjetas PCIe $\times 1$, $\times 4$, $\times 8$ y $\times 16$ [8]. El enlace con Raspberry Pi 5 continúa siendo, en cualquier caso, PCIe $\times 1$.

No se conserva el número de artículo exacto del adaptador utilizado. La identificación disponible del montaje se recoge en el cuadro A.1.

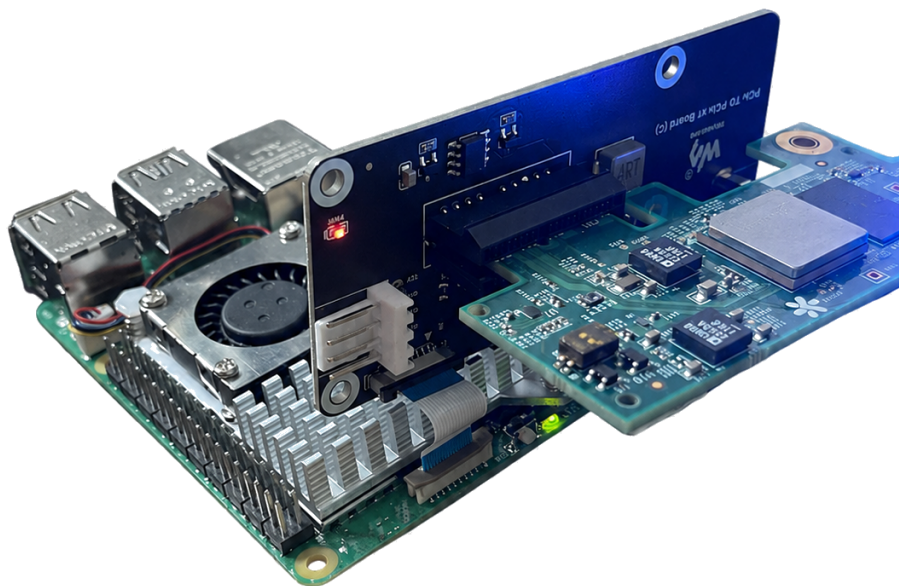


Figura 2.2: Montaje utilizado durante una de las sesiones del TFG.

2.5 Comprobación antes del encendido

Antes de alimentar el montaje, comprueba que el cable FFC está correctamente orientado, que la tarjeta está bien insertada y que las tensiones de alimentación coinciden con las indicadas para los componentes utilizados.

Condición de parada H0

Si no puedes confirmar la tensión, la polaridad o la orientación de alguna conexión, no alimentes el montaje hasta comprobarlo en la documentación correspondiente.

3 Preparación del host y controlador PCIe

3.1 Comprobaciones iniciales

Antes de instalar el controlador, comprueba la versión del sistema, la arquitectura y el kernel. En Linux puedes guardar esta información en la carpeta `evidence`:

```
mkdir -p evidence
{
  cat /etc/os-release
  uname -r
  uname -m
  lspci -nn
} | tee evidence/host-inventory.txt
```

`uname -m` suele mostrar `x86_64` en un PC y `aarch64` en una Raspberry Pi 5 con un sistema de 64 bits. La versión del kernel será necesaria para comprobar la compatibilidad del controlador.

3.2 Habilitar PCIe en Raspberry Pi 5

Esta sección solo es necesaria si utilizas una Raspberry Pi 5. Los dispositivos HAT+ compatibles pueden detectarse automáticamente, mientras que para un adaptador PCIe no HAT+ Raspberry Pi indica que debe habilitarse la interfaz PCIe en el archivo de configuración [7].

En Raspberry Pi OS actual, abre:

```
sudoedit /boot/firmware/config.txt
```

Si esa ruta no existe, comprueba `/boot/config.txt`. Añade:

```
dtparam=pciex1
```

Guarda el archivo y reinicia:

```
sudo reboot
```

Raspberry Pi 5 utiliza PCIe Gen2 por defecto. Aunque es posible habilitar Gen3, Raspberry Pi advierte que la placa no está certificada para funcionar de forma estable a esa velocidad [7]. Para la puesta en marcha se mantiene Gen2.

3.3 Comprobar la conexión PCIe

Después del arranque, comprueba que el sistema detecta la tarjeta:

```
lspci -nn
```

Debe aparecer un dispositivo PCIe correspondiente a la tarjeta. Si no aparece, revisa el montaje, la alimentación y, en Raspberry Pi 5, que PCIe esté habilitado antes de continuar con el controlador.

Si necesitas más información sobre el dispositivo detectado, utiliza:

```
lspci -nnk
```

En este punto solo se está comprobando la conexión PCIe. Que la tarjeta aparezca en `lspci` no significa todavía que el controlador Akida esté instalado.

3.4 Instalar el controlador

El controlador PCIe oficial de BrainChip se distribuye en el repositorio `akida_dw_edma`. Antes de instalarlo, comprueba la versión del kernel:

```
uname -r
```

La versión del controlador consultada para este manual admite kernels desde 5.4 hasta 6.8. En kernels 6.9 o posteriores el módulo no se compila [3]. Como esta compatibilidad puede cambiar, consulta también el README del repositorio antes de instalarlo.

Kernel no compatible

Si la versión del kernel queda fuera del intervalo soportado por el controlador, no continúes con la instalación hasta comprobar si BrainChip ha publicado una versión compatible.

En Ubuntu, instala las dependencias indicadas por BrainChip:

```
sudo apt update
sudo apt install dkms build-essential "linux-headers-$(uname -r)"
```

En otras distribuciones los nombres de los paquetes pueden cambiar, pero deben instalarse DKMS, las herramientas de compilación y los headers correspondientes al kernel en uso.

A continuación, clona el repositorio e instala el controlador:

```
git clone https://github.com/Brainchip-Inc/akida_dw_edma.git
cd akida_dw_edma
git rev-parse HEAD
./install.sh
```

La salida de `git rev-parse HEAD` registra la revisión exacta del controlador utilizada; también puedes usar una configuración propia.

3.5 Comprobar el controlador

Después de instalar el controlador, revisa el estado de DKMS y confirma que el dispositivo Akida está disponible:

```
dkms status
lsmod | grep -i akida
ls -l /dev/akida*
lspci -nnk
```

La instalación puede considerarse correcta cuando el dispositivo sigue apareciendo en PCIe, el módulo está cargado y existe al menos un nodo `/dev/akida*`.

Si el dispositivo aparece en `lspci` pero falta `/dev/akida*`, consulta los mensajes del kernel:

```
sudo dmesg | grep -iE 'akida|edma|pcie' | tail -n 120
```

Si el nodo existe pero solo puede utilizarse como `root`, revisa sus permisos y la regla `udev` instalada por el controlador antes de continuar.

Con el controlador funcionando, continúa con la configuración del entorno de software del capítulo 4.

4 Entorno de software y primera prueba

4.1 Preparar el entorno

BrainChip distribuye las herramientas de Akida mediante MetaTF. En la versión utilizada para este manual, la instalación actual emplea `metatf==2.19.3` y admite Python 3.10–3.12 [5].

El repositorio también incluye una configuración de referencia basada en Python 3.11 y Akida 2.19.1, la misma versión utilizada durante el TFG. Esta segunda opción permite repetir la prueba básica con un entorno conocido.

Abre una terminal en `akd1000-bringup`, la carpeta extraída del ZIP.

En Linux:

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

En Windows, PowerShell:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

El entorno virtual solo se crea una vez. En una terminal nueva basta con volver a activarlo antes de continuar.

4.2 Instalar Akida

Elige una de las dos configuraciones siguientes.

Para trabajar con la versión actual documentada por BrainChip:

```
python -m pip install "metatf==2.19.3"
```

Para repetir la configuración de referencia incluida con el repositorio:

```
python -m pip install -r requirements/reproducible.txt
```

No mezcles ambas instalaciones en el mismo entorno virtual.

A continuación, instala las herramientas incluidas con este repositorio:

```
python -m pip install -e . --no-deps
python -m pip check
```

Comprueba que Akida se importa correctamente:

```
python -c "import akida; print(akida.__version__); print(akida.devices())"
```

Si no hay una tarjeta conectada, es normal que `akida.devices()` devuelva []. Si el equipo tiene una AKD1000 conectada y el controlador está funcionando, debe aparecer al menos un dispositivo.

4.3 Primera prueba en software

Antes de utilizar la tarjeta, ejecuta el modelo de prueba incluido con el repositorio:

```
python examples/00_simulator_smoke.py
```

El modelo recibe cuatro valores de entrada. Para

```
[3, 2, 1, 4]
```

la salida esperada es

```
[5, 5]
```

El script ejecuta primero el modelo en software y comprueba después que puede mapearse sobre un AKD1000 virtual. Este último paso verifica la compatibilidad del modelo con la arquitectura de la tarjeta, pero no utiliza hardware físico.

También puedes ejecutar la comprobación incluida en la herramienta del repositorio:

```
akd1000-bringup verify --id smoke --model models/akida_v1_smoke_test.fbz
```

Si ambas pruebas terminan correctamente, el entorno de Python y el modelo de prueba están preparados.

4.4 Comprobar el equipo

Antes de usar la tarjeta física, puedes ejecutar el diagnóstico incluido con el repositorio:

```
akd1000-bringup doctor
```

Para guardar el resultado:

```
mkdir -p evidence
akd1000-bringup doctor --json > evidence/preflight.json
```

La herramienta comprueba, entre otros datos, la versión de Akida, el kernel, la conexión PCIe y los dispositivos detectados. Su salida ayuda a localizar el problema si la tarjeta no aparece.

En un equipo sin AKD1000 es normal que el diagnóstico indique que no se ha encontrado ningún dispositivo.

4.5 Primera ejecución en la AKD1000

Esta prueba requiere que la tarjeta haya sido detectada por PCIe y que el controlador del capítulo 3 esté instalado.

Comprueba primero que Akida detecta el dispositivo:

```
akida devices
```

Después ejecuta:

```
python examples/01_hardware_smoke.py
```

El script carga el mismo modelo utilizado en la prueba de software, lo mapea sobre la AKD1000 y ejecuta la entrada conocida. La salida debe volver a ser:

[5, 5]

Un resultado `PASS` confirma que el modelo se ha cargado, que la tarjeta es accesible desde Akida y que la inferencia se ha ejecutado correctamente en el hardware.

Si la prueba física falla

No continúes todavía con un modelo propio. Conserva el mensaje de error y utiliza el procedimiento de diagnóstico del [capítulo 7](#) para localizar el problema.

5 Operación de modelos Akida

5.1 Cargar e inspeccionar un modelo

Los modelos Akida se distribuyen como archivos FBZ. Antes de ejecutarlos conviene conocer, al menos, la forma de entrada, la forma de salida y el preprocesado utilizado durante su conversión.

Carga el modelo y revisa su estructura:

```
import akida

model = akida.Model("models/model.fbz")

print("input_shape:", model.input_shape)
print("output_shape:", model.output_shape)
print("ip_version:", model.ip_version)
model.summary()
```

`summary()` muestra las capas y la forma en la que el modelo puede ejecutarse. La cantidad de recursos utilizados depende del propio modelo y del modo de mapeo.

5.2 Seleccionar el dispositivo

Los dispositivos físicos disponibles se obtienen mediante `akida.devices()` [4]:

```
import akida

devices = list(akida.devices())

if not devices:
    raise RuntimeError("No se ha detectado ningun dispositivo Akida")

device = devices[0]
print(device)
```

Un dispositivo creado con `akida.AKD1000()` es virtual y sirve para comprobar el mapeo de un modelo. Para ejecutar una inferencia en la tarjeta es necesario utilizar uno de los dispositivos devueltos por `akida.devices()`.

5.3 Mapear el modelo en la tarjeta

Un modelo cargado puede ejecutarse inicialmente mediante software. Para utilizar la AKD1000 debe mapearse sobre el dispositivo:

```

model.map(
    device,
    hw_only=True,
    mode=akida.MapMode.AllNps
)

model.summary()

```

Con `hw_only=True` se exige que la ejecución se realice completamente en el hardware Akida [4]. Si el modelo no puede mapearse de esta forma, la llamada produce un error.

`AllNps`, `HwPr` y `Minimal` son distintos modos de asignar los recursos de la AKD1000. Para una primera ejecución puedes usar `AllNps`. Su efecto sobre latencia y consumo se estudia en el capítulo 6.

5.4 Preparar la entrada

La forma y el tipo de los datos dependen del modelo. Antes de ejecutar una inferencia, comprueba que la entrada coincide con `model.input_shape`.

Por ejemplo, para datos almacenados en NumPy:

```

import numpy as np

data = np.load("data/inputs.npy", allow_pickle=False)

print("shape:", data.shape)
print("dtype:", data.dtype)
print("min/max:", data.min(), data.max())

if tuple(data.shape[1:]) != tuple(model.input_shape):
    raise ValueError(
        f"Forma de entrada {data.shape[1:]} "
        f"!= {model.input_shape}"
    )

```

El tipo, el rango y el orden de los canales deben ser los mismos utilizados durante la preparación y conversión del modelo. No existe un rango de entrada común para todos los modelos Akida.

En particular, el rango 0–15 utilizado por los modelos AoA del TFG pertenece a ese caso concreto y no debe aplicarse a otros modelos.

5.5 Ejecutar el modelo

La API de Akida permite ejecutar el modelo mediante `forward()` o `predict()`. La elección depende de cómo se haya definido la salida del modelo y de cómo se haya validado durante su conversión [4].

Por ejemplo:

```

output = model.forward(data)
print(output)

```

Durante una campaña de medidas debe mantenerse el mismo método de ejecución para que los resultados sean comparables.

También puedes usar la herramienta incluida en este repositorio:

```
akd1000-bringup run \  
  --model models/model.fbz \  
  --input data/input.npy \  
  --method forward \  
  --clock Performance \  
  --map AllNps \  
  --output outputs/raw-output.npy
```

La herramienta carga la entrada, ejecuta el modelo y guarda la salida en un archivo NumPy. La entrada debe tener la forma

```
(batch, *model.input_shape)
```

Los modelos con varias entradas o salidas requieren tratarlos directamente mediante la API de Akida.

5.6 Modo de reloj

La AKD1000 dispone de los modos de reloj Performance, Economy y LowPower. El modo puede seleccionarse mediante la API:

```
device.soc.clock_mode = akida.soc.ClockMode.Performance  
print(device.soc.clock_mode)
```

El modo de reloj debe anotarse cuando se comparen medidas de rendimiento. Un modo de menor potencia instantánea no implica necesariamente una menor energía por inferencia, ya que también puede cambiar el tiempo de ejecución.

6 Medida de rendimiento y energía

6.1 Qué se está midiendo

Antes de comparar resultados, mantén fijos el modelo, los datos y la configuración de ejecución. Anota también el modo de reloj, el modo de mapeo y el número de repeticiones.

6.2 Medida de latencia

La herramienta mide el tiempo empleado por la llamada de ejecución del modelo. La medida incluye la comunicación entre el host y la tarjeta, la llamada al SDK y la inferencia en la AKD1000.

La carga de los archivos, el mapeo del modelo y las ejecuciones de calentamiento se realizan antes de comenzar a medir.

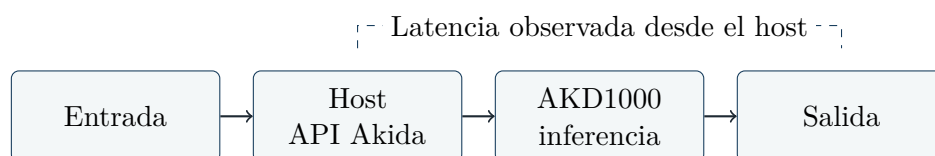


Figura 6.1: Elementos incluidos en la medida de latencia realizada desde el host.

Por tanto, los resultados de este manual se expresan como *latencia observada desde el host*. No corresponden a un temporizador interno aislado del AKD1000.

Si se quiere medir el tiempo completo de una aplicación, deben añadirse también las etapas de adquisición, preprocesado y postprocesado.

6.3 Ejecutar un benchmark

Antes de una campaña completa, ejecuta una prueba corta para comprobar que el modelo, los datos y la configuración son correctos.

```
akd1000-bringup benchmark \
  --model models/model.fbz \
  --input data/inputs.npy \
  --method forward \
  --clock Performance \
  --map AllNps \
  --warmup 8 \
  --repetitions 1 \
  --output outputs/smoke
```

La carpeta de salida contiene:

- `latencies.csv`, con las medidas individuales;
- `summary.json`, con la configuración y el resumen del ensayo;

- `host.json`, con información del sistema utilizado.

Comprueba que el número de muestras, las unidades y la configuración son los esperados. Si la prueba es correcta, repite el benchmark con el número de repeticiones necesario y guarda los resultados en una carpeta distinta.

Para comparar configuraciones, cambia una sola variable cada vez. Por ejemplo, puedes mantener fijo el modelo y comparar los modos `AllNps`, `HwPr` y `Minimal`, o mantener fijo el mapeo y cambiar el modo de reloj.

6.4 Potencia y energía

Akida permite habilitar la medida de potencia del dispositivo y consultar las estadísticas después de una inferencia [4]:

```
device.soc.power_measurement_enabled = True

model.forward(data)

print(model.statistics)
```

La potencia proporcionada por esta API corresponde a la medida disponible en el dispositivo. No representa el consumo completo de la Raspberry Pi, el carrier y la tarjeta.

Con una potencia media y la latencia medida desde el host, puede calcularse una energía estimada por inferencia como

$$E_{\text{est}} [\mu\text{J}] = P_{\text{API}} [\text{mW}] t_{\text{host}} [\text{ms}]. \quad (6.1)$$

La relación de unidades es directa:

$$1 \text{ mW} \cdot 1 \text{ ms} = 1 \mu\text{J}.$$

Esta energía es útil para comparar ejecuciones realizadas con el mismo procedimiento, pero no debe interpretarse como el consumo energético total del sistema.

Para medir el consumo completo sería necesario utilizar instrumentación externa sobre la alimentación del montaje.

6.5 Guardar los resultados

Para cada ensayo conserva, como mínimo:

- el modelo y su versión;
- los datos utilizados;
- el modo de reloj;
- el modo de mapeo;
- el número de repeticiones;
- los archivos generados por el benchmark.

Si se comparan distintas plataformas, como CPU, GPU o FPGA, deben utilizarse la misma tarea, los mismos datos y una definición equivalente de la medida.

7 Diagnóstico

Cuando aparece un error, conviene comprobar el sistema en el mismo orden seguido durante la puesta en marcha: conexión PCIe, controlador, entorno de Python, modelo y datos. La [cuadro 7.1](#) resume los fallos más habituales y dónde empezar a buscarlos.

Cuadro 7.1: Guía rápida de diagnóstico.

Síntoma	Comprueba	Qué hacer
La tarjeta no aparece en <code>lspci</code> .	Alimentación, cable FFC, montaje y configuración PCIe de la Raspberry Pi 5.	Revisa primero la conexión física y la configuración PCIe.
La tarjeta aparece en PCIe, pero no existe <code>/dev/akida*</code> .	Versión del kernel, headers, <code>dkms status</code> , <code>lsmod</code> y mensajes de <code>dmesg</code> .	Revisa la instalación y compatibilidad del controlador.
Existe <code>/dev/akida*</code> , pero <code>akida.devices()</code> está vacío.	Entorno virtual activo, versión de Akida y permisos del dispositivo.	Comprueba que estás utilizando el entorno correcto y que el usuario puede acceder al dispositivo.
<code>import akida</code> produce un error.	Versión de Python y paquetes instalados en el entorno activo.	Activa el entorno virtual correcto o reinstala Akida dentro de él.
El archivo FBZ no carga.	Ruta del archivo, versión de Akida y compatibilidad del modelo.	Comprueba primero el modelo de prueba incluido con el repositorio.
El modelo no puede mapearse con <code>hw_only=True</code> .	Salida de <code>model.summary()</code> , capas utilizadas y versión del modelo.	Comprueba que el modelo es compatible con la ejecución completa en la AKD1000.
La entrada es rechazada.	Forma, tipo, rango y orden de los datos.	Compara la entrada con <code>model.input_shape</code> y con el preprocesado utilizado durante la conversión.
La ejecución termina, pero la salida no es la esperada.	Preprocesado, orden de las muestras y uso de <code>forward()</code> o <code>predict()</code> .	Prueba primero una entrada cuyo resultado esperado sea conocido.
La latencia cambia mucho entre ejecuciones.	Modo de reloj, modo de mapeo, calentamiento, carga del host y número de repeticiones.	Repite la medida manteniendo fija la configuración.
No aparece una medida de potencia.	Medición habilitada y estadísticas devueltas por Akida.	Comprueba la configuración y aumenta la duración de la prueba si es necesario.

7.1 Información útil para diagnosticar un fallo

La herramienta incluida con el repositorio puede recoger buena parte de la información necesaria para localizar un problema:

```
mkdir -p evidence
akd1000-bringup doctor --json > evidence/doctor.json
```

Si necesitas más detalle, guarda también el estado del sistema y del controlador:

```
uname -a > evidence/uname.txt
lspci -nnk > evidence/lspci-nnk.txt
dkms status > evidence/dkms.txt
ls -l /dev/akida* > evidence/device-nodes.txt 2>&1
python -m pip freeze > evidence/pip-freeze.txt
sudo dmesg | grep -iE 'akida|edma|pcie' > evidence/driver-log.txt
```

Cuando informes de un problema, conserva también el comando que produjo el error y el mensaje completo. Con esa información suele ser posible distinguir si el fallo está en PCIe, el controlador, el entorno de Python, el modelo o los datos.

8 Mantenimiento y actualización

8.1 Guardar una configuración conocida

Después de completar correctamente la primera ejecución en la tarjeta, guarda la información básica del entorno. Servirá como referencia si una actualización provoca algún problema.

```
mkdir -p evidence/known-good

akd1000-bringup doctor --json \
> evidence/known-good/doctor.json

python -m pip freeze \
> evidence/known-good/pip-freeze.txt

sha256sum models/*.fbz \
> evidence/known-good/model-hashes.txt
```

Conserva también la versión del proyecto y del controlador utilizados. Si más adelante el sistema deja de funcionar, esta configuración permite comparar qué ha cambiado.

8.2 Actualizar el entorno

Las actualizaciones del kernel, del controlador, de Python o de Akida pueden afectar al funcionamiento de la tarjeta. Siempre que sea posible, cambia un componente cada vez y repite después la prueba básica.

Un procedimiento sencillo es:

1. comprueba la compatibilidad de la nueva versión;
2. realiza la actualización;
3. reinicia el equipo si es necesario;
4. repite la detección de la tarjeta y las pruebas de software y hardware.

Si aparece un problema, compara el sistema con la configuración guardada anteriormente y utiliza el procedimiento del [capítulo 7](#).

El controlador utiliza DKMS para reconstruir el módulo después de una actualización del kernel. Esto solo funciona mientras la nueva versión del kernel siga estando soportada por el controlador.

8.3 Apagado y almacenamiento

Antes de desconectar la tarjeta, termina las ejecuciones en curso y apaga el host de forma normal. Después, desconecta las fuentes de alimentación antes de retirar la tarjeta o el cable FFC.

Si el montaje va a guardarse durante un periodo largo, protege las placas frente a descargas electrostáticas y evita doblar o dejar bajo tensión el cable FFC.

A Caso documentado del TFG

Este apéndice resume el montaje, el entorno de software y los resultados utilizados durante el TFG. Corresponden a una configuración concreta y no deben interpretarse como requisitos generales de la AKD1000.

A.1 Montaje utilizado

El montaje empleado durante el TFG estaba formado por los componentes recogidos en el cuadro A.1 [6].

Cuadro A.1: Hardware utilizado durante el TFG.

Elemento	Identificación	Observaciones
Host	Raspberry Pi 5 Model B, 8 GB de RAM, Rev 1.1.	Adquirida dentro de un kit db-tronic.
Almacenamiento	SSD NVMe M.2 de 64 GB.	Incluido en el kit; no se conserva el modelo exacto.
Refrigeración y carcasa	Sistema de refrigeración activa y carcasa incluidos en el kit.	No se conserva la referencia comercial.
Fuente del host	Fuente de alimentación de 27 W.	Incluida en el kit.
Adaptador	Waveshare <i>PCIe TO PCIe x1 Board (C)</i> para Raspberry Pi 5.	Conectado a la Raspberry Pi mediante FFC.
Cable	FFC con serigrafía <i>PCIe-Cable-50mm</i> .	Longitud de 50 mm.
Tarjeta	BrainChip Akida PCIe Board AKD1000.	El dispositivo utilizado aparecía como BC.00.000.002.
Alimentación de la tarjeta	12 V en continua mediante el carrier.	Alimentación independiente de la Raspberry Pi.

La cadena física utilizada fue:

Raspberry Pi 5 → cable FFC → carrier Waveshare → BrainChip AKD1000

La Raspberry Pi se alimentó mediante USB-C y la tarjeta AKD1000 mediante la entrada de 12 V del carrier.

A.2 Entorno de software

Las versiones utilizadas durante la campaña final se muestran en el [cuadro A.2](#).

Cuadro A.2: Entorno de software del TFG.

Elemento	Versión
Sistema operativo	Debian GNU/Linux 12 (bookworm)
Python	3.11.2
TensorFlow	2.19.1
API Keras	<code>tf_keras</code>
Akida	2.19.1
CNN2SNN	2.19.1
QuantizeML	1.2.3
Dispositivo	BC.00.000.002
Configuración del host	<code>governor performance; throttled=0x0</code>

Estas versiones corresponden al entorno utilizado en el experimento. Para reproducir completamente los resultados también son necesarios los modelos, los datos y el mismo procedimiento de medida.

A.3 Modelos AoA

El TFG utilizó dos redes para estimar el ángulo de llegada (*Angle of Arrival*, AoA). Cada muestra contenía ocho valores enteros de intensidad de envolvente entre 0 y 15.

La salida del modelo contenía 180 valores, uno por cada clase. La clase c correspondía al ángulo

$$\theta = 2c. \quad (\text{A.1})$$

Los dos modelos finales fueron:

Cuadro A.3: Modelos AoA utilizados en el TFG.

Modelo	Arquitectura	Pesos sin sesgos	Uso
Compacto	$8 \rightarrow 10 \rightarrow 180$	1.880	Modelo de baja complejidad
Principal	$8 \rightarrow 957 \rightarrow 180$	179.916	Modelo con mayor precisión

El error angular se calculó teniendo en cuenta la naturaleza circular del ángulo:

$$e_{\text{circ}} = ((\hat{\theta} - \theta + 180) \bmod 360) - 180. \quad (\text{A.2})$$

Por ejemplo, una predicción de 0° para una referencia de 358° produce un error de 2° y no de -358° .

Los modelos FBZ y el dataset utilizados en la campaña original no forman parte de esta entrega. El modelo de prueba incluido con el repositorio se utiliza únicamente para comprobar el funcionamiento del entorno y de la tarjeta.

A.4 Resultados

La evaluación se realizó sobre 576 muestras. El modelo principal obtuvo un MAE angular de $4,09^\circ$, frente a $9,13^\circ$ del modelo compacto [6].

En las configuraciones estudiadas, los puntos de menor energía estimada fueron:

- modelo compacto: Economy + HwPr, con 0,565 ms, 770,0 mW y $435,2 \mu\text{J}$;
- modelo principal: Performance + AllNps, con 0,756 ms, 731,8 mW y $553,3 \mu\text{J}$.

		Modelo compacto — Latencia (ms)			Modelo compacto — Energía estimada (μJ)		
ClockMode	Performance	0.553	0.543	0.544	515.2	506.3	507.6
	Economy	0.569	0.565	0.569	439.1	435.2	438.2
	LowPower	1.287	1.281	1.263	890.5	885.5	873.0
		AllNps	HwPr MapMode	Minimal	AllNps	HwPr MapMode	Minimal
		Modelo principal — Latencia (ms)			Modelo principal — Energía estimada (μJ)		
ClockMode	Performance	0.756	0.842	0.774	553.3	682.8	690.2
	Economy	1.069	1.056	1.023	951.2	865.4	789.4
	LowPower	10.228	10.250	10.279	7091.7	7071.2	7082.8
		AllNps	HwPr MapMode	Minimal	AllNps	HwPr MapMode	Minimal

Figura A.1: Latencia y energía estimada para los dos modelos en las nueve combinaciones de modo de reloj y mapeo evaluadas durante el TFG.

La latencia se midió desde el host alrededor de la llamada de ejecución. La potencia corresponde a la medida proporcionada por la API de Akida y no al consumo completo de la Raspberry Pi, el carrier y la tarjeta.

Estos resultados describen únicamente la plataforma utilizada durante el TFG.

B Referencia rápida de comandos

Este apéndice reúne los comandos principales del procedimiento en Linux. Las explicaciones y condiciones de uso se encuentran en los capítulos correspondientes.

B.1 Comprobar el sistema

```
uname -r
uname -m
cat /etc/os-release
lspci -nn
```

B.2 Raspberry Pi 5

Para un adaptador PCIe no HAT+, edita:

```
sudoedit /boot/firmware/config.txt
```

y añade, si todavía no está presente:

```
dtparam=pciex1
```

Después reinicia y comprueba la conexión:

```
sudo reboot
lspci -nn
```

B.3 Controlador PCIe

En Ubuntu:

```
sudo apt update
sudo apt install dkms build-essential "linux-headers-$(uname -r)"
```

Descarga e instala el controlador:

```
git clone https://github.com/Brainchip-Inc/akida_dw_edma.git
cd akida_dw_edma
git rev-parse HEAD
./install.sh
```

Comprueba la instalación:

```
dkms status
lsmod | grep -i akida
ls -l /dev/akida*
lspci -nnk
```

B.4 Entorno de Python

Desde la carpeta principal del proyecto:

```
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
```

Instalación actual de MetaTF:

```
python -m pip install "metatf==2.19.3"
```

o entorno de referencia del repositorio:

```
python -m pip install -r requirements/reproducible.txt
```

Instala después las herramientas del proyecto:

```
python -m pip install -e . --no-deps
python -m pip check
```

B.5 Pruebas básicas

Prueba en software:

```
python examples/00_simulator_smoke.py
akd1000-bringup verify \
  --id smoke \
  --model models/akida_v1_smoke_test.fbz
```

Diagnóstico:

```
akd1000-bringup doctor
```

Prueba en la tarjeta:

```
akida devices
python examples/01_hardware_smoke.py
```

B.6 Ejecutar un modelo propio

```
akd1000-bringup run \
  --model models/model.fbz \
  --input data/input.npy \
  --method forward \
  --clock Performance \
  --map AllNps \
  --output outputs/raw-output.npy
```

B.7 Benchmark

```
akd1000-bringup benchmark \  
  --model models/model.fbz \  
  --input data/inputs.npy \  
  --method forward \  
  --clock Performance \  
  --map AllNps \  
  --warmup 8 \  
  --repetitions 1 \  
  --output outputs/smoke
```

B.8 Diagnóstico rápido

1. Si la tarjeta no aparece en `lspci`, revisa el montaje y la configuración PCIe.
2. Si aparece en PCIe pero falta `/dev/akida*`, revisa el controlador.
3. Si existe el nodo pero `akida.devices()` está vacío, revisa el entorno y los permisos.
4. Si el modelo de prueba funciona y el modelo propio no, revisa el modelo y sus datos de entrada.

C Fuentes y limitaciones

Las instrucciones de este manual se basan principalmente en la documentación oficial de Brain-Chip, Raspberry Pi y Waveshare, junto con la memoria y los archivos del TFG.

La versión de referencia del entorno utiliza Akida 2.19.1, mientras que la documentación actual consultada para este manual corresponde a MetaTF 2.19.3. La compatibilidad del controlador PCIe debe comprobarse antes de utilizar versiones posteriores del kernel o del SDK.

Esta edición todavía no ha sido validada mediante una sesión completa con un usuario nuevo y acceso físico a la tarjeta. La reproducción del caso AoA requiere además los modelos y datos originales, que no forman parte de esta entrega.

Las medidas de potencia obtenidas mediante la API de Akida no representan el consumo total del sistema. El manual tampoco cubre funciones como *edge learning* ni Akida Engine.

Bibliografía

- [1] BrainChip, Inc. *AKD1000 Akida System-on-Chip Product Brief*, August 2025. Versión 2.3, agosto de 2025. <https://brainchip.com/wp-content/uploads/2025/08/Akida-AKD1000-SoC-Product-Brief-V2.3-Aug.25.pdf>.
- [2] BrainChip, Inc. *AKD1000 PCIe Card Hardware Product Brief*, 2025. Product Brief v1.0. <https://brainchip.com/wp-content/uploads/2025/04/AKD1000-PCIe-Card-Product-Brochure-v1.0.pdf>.
- [3] BrainChip, Inc. Akida pcie driver: akida_dw_edma. Repositorio de código, 2026. Rama master consultada el 29 de septiembre de 2026. https://github.com/Brainchip-Inc/akida_dw_edma.
- [4] BrainChip, Inc. *Akida User Guide*, 2026. MetaTF 2.19.3; Consultado el 29 de septiembre de 2026. https://doc.brainchipinc.com/user_guide/akida.html.
- [5] BrainChip, Inc. *MetaTF Documentation: Installation*, 2026. Consultado el 29 de septiembre de 2026. <https://doc.brainchipinc.com/installation.html>.
- [6] Álvaro Poveda Ruiz. Aceleración y despliegue de redes neuronales mediante hardware neuromórfico para procesamiento radar. Trabajo fin de grado, Universidad Politécnica de Madrid, 2026.
- [7] Raspberry Pi Ltd. *Raspberry Pi Computer Hardware: PCIe Connector*, 2026. Consultado el 29 de septiembre de 2026. <https://www.raspberrypi.com/documentation/computers/raspberry-pi.html#raspberry-pi-connector-for-pcie>.
- [8] Waveshare. *PCIe TO PCIe x1 Board (C)*, 2026. Guía de conexión consultada el 29 de septiembre de 2026. [https://www.waveshare.com/wiki/PCIe_TO_PCIe_x1_Board_\(C\)](https://www.waveshare.com/wiki/PCIe_TO_PCIe_x1_Board_(C)).